

Practical 2: Breadth-First Search (BFS) Algorithm

Objective

To understand, implement, and analyze the Breadth-First Search algorithm for graph traversal and pathfinding problems.

Theory

What is Breadth-First Search?

Breadth-First Search (BFS) is a graph traversal algorithm that explores vertices in layers, visiting all vertices at the current depth before moving to vertices at the next depth level. It uses a queue data structure to maintain the order of exploration.

Algorithm Characteristics:

- **Time Complexity:** $O(V + E)$ where V is vertices and E is edges
- **Space Complexity:** $O(V)$ for the queue and visited array
- **Completeness:** Always finds a solution if one exists
- **Optimality:** Finds the shortest path in unweighted graphs

BFS Algorithm Steps:

1. Start with a source vertex and mark it as visited
2. Add the source vertex to a queue
3. While the queue is not empty:
 - Dequeue a vertex from the front of the queue
 - For each unvisited neighbor of the current vertex:
 - o Mark the neighbor as visited
 - o Add the neighbor to the queue
 - o Set the parent of the neighbor (for path reconstruction)

Applications:

- Finding shortest path in unweighted graphs
- Level-order traversal of trees
- Web crawling
- Social network analysis
- GPS navigation systems

BREADTH FIRST SEARCH

```
In [1]: import networkx as nx
import matplotlib.pyplot as plt
```

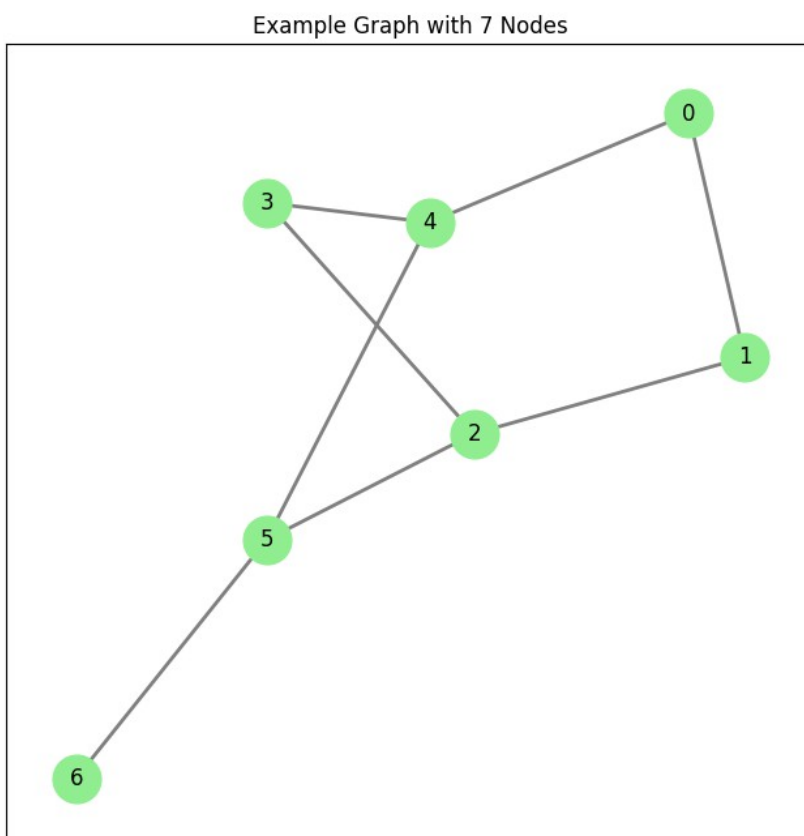
```
G = nx.Graph()
G.add_nodes_from(range(7))

edges = [
    (0, 1),
    (1, 2),
    (2, 3),
    (3, 4),
    (4, 5),
    (5, 6),
    (0, 4),
    (2, 5)
]

num_nodes = G.number_of_nodes()
G.add_edges_from(edges)
pos = nx.spring_layout(G)
```

```
In [2]: plt.figure(figsize=(8, 8))
nx.draw_networkx_nodes(G, pos, node_color='lightgreen', node_size=700)
nx.draw_networkx_edges(G, pos, width=2, edge_color='gray')
nx.draw_networkx_labels(G, pos, font_size=12, font_family='sans-serif')

# Set the title and display the plot.
plt.title("Example Graph with 7 Nodes")
plt.show()
```



In [3]: `import numpy as np`

```
class graph:
    def __init__(self , num_nodes , edges):
self.num_nodes = num_nodes
self.edges = edges
self.data = [[] for _ in range(num_nodes)]
        for n1, n2 in edges:
            self.data[n1].append(n2)
self.data[n2].append(n1)

    def __repr__(self):
        return "\n".join(["{} : {}".format(n,x) for n,x in enumerate(self.data)])

    def __str__(self):
        return self.__repr__()

    def add_edge(self, n1, n2):
        if self.num_nodes is not None :
edges.append((n1,n2)) for n1, n2 in
edges:
            if n2 not in self.data[n1]:
self.data[n1].append(n2) if n1 not in
self.data[n2]:
self.data[n2].append(n1)
    def mat(self):
array = np.zeros(
self.num_nodes*self.num_nodes
).reshape(self.num_nodes,self.num_nodes) for
(x,y) in self.edges :

array[x][y]=1 return array

    def bfs(self , root):
quene = []
discovered = [False] * len(self.data) distance =
[None] * len(self.data)
parent = [None] * len(self.data)
distance[root] = 0
discovered[root] = True
quene.append(root)
idx = 0

        while idx < len(quene) :
current = quene[idx]
idx += 1

            for node in self.data[current]: if not discovered[node]:
distance[node] = 1 + distance[current]

parent[node] = current
discovered[node] = True quene.append(node)

        return quene , distance , parent

    def check_connected_nodes(self):
quene,_ ,_ = self.bfs(self.edges[0][0])
if len(quene) == self.num_nodes:
return True
else :
return False
```

```
In [4]: g1 = graph(num_nodes, edges)
        print(g1)
```

```
0 : [1, 4]
1 : [0, 2]
2 : [1, 3, 5]
3 : [2, 4]
4 : [3, 5, 0]
5 : [4, 6, 2]
6 : [5]
```

```
In [5]: g1.add_edge(2,6)
        print(g1)
```

```
0 : [1, 4]
1 : [0, 2]
2 : [1, 3, 5, 6]
3 : [2, 4]
4 : [3, 5, 0]
5 : [4, 6, 2]
6 : [5, 2]
```

```
In [6]: g1.bfs(3)
```

```
Out[6]: ([3, 2, 4, 1, 5, 6, 0], [2, 2, 1, 0, 1, 2, 2], [4, 2, 3, None, 3, 2, 2])
```

```
In [7]: g1.check_connected_nodes()
```

```
Out[7]: True
```